server can use that identification to remember the user's state for each request. Over time, interaction patterns were created where nearly all state would be managed on the server, and the browser's job was largely to ask for new pages or new parts of pages, receive the result of the state change, and display it to the user.

Designing Around Basic Web Actions

Ruby on Rails is a framework for managing the state of a web application on the server. Rails is, to a large extent, built around the idea that most web interactions involve a very small set of operations. This set of actions is often abbreviated CRUD: Create, Read, Update, and Delete. In Rails, these actions are represented by the seven operations provided for a standard resource (create, new, show, index, edit, update, and delete).

One of the great insights of Rails is that once you've settled on this set of basic actions, you can remove a lot of repetitive boilerplate and assume common behavior no matter what shape the data is in. In Rails, this default behavior is often defined by the scaffolding that Rails creates for new resources.

If you are dealing with these basic actions, it turns out web browsers can offer you a lot of help. Browsers can provide data input elements, manage the state of form elements, and maintain a list of historical actions. Working hand in hand with a browser makes the Rails' core actions more powerful.

In fact, the basic set of Rails interactions is so powerful that it starts to be worthwhile to take things that are not necessarily basic resource interactions and model them as if they were. Take, for example, Twitter. Twitter, which was originally built partially using Rails, can be modeled as a system where a tweet is a resource, and the user has actions to create one, show one or more, and delete one.[23] (But not edit, which is an argument I'm not getting into here.) Is that the best way to model Twitter's user interaction? I don't know. Probably not. But it's at least a pretty good way to model Twitter, and doing so gives you a big head start because you can take advantage of the power of Rails and the browsers.